

N-Body Simulation in RISC-V Assembly

Milestone 2 — Caal 100494

Group Project

Computer Architecture and Assembly Language

Arwa Mahmood (31631) — Eshaal Gardezi (30868) — Ayesha Arif (31597)

I- Introduction

The goal was to simulate gravitational interactions between N celestial bodies (`solar300.csv`) using the leapfrog Kick-Drift-Kick algorithm, implemented entirely in RISC-V assembly. The simulation writes particle positions each iteration to a memory-mapped file for realtime Python GUI visualization.

II- Refactoring simple_example.py

The original Python code used high-level features with no assembly equivalent, dynamic lists with `.append()` and the `**` operator for powers. We created `refactored_simple_example.py` using only flat pre-allocated arrays:

```
p_x = [0.0] * N
p_y = [0.0] * N
p_z = [0.0] * N
v_x = [0.0] * N # etc.
```

This was the first version, no real data, just verifying the algorithm structure with flat arrays. Math was simplified: `dx**2` became `dx*dx`, and `dist**0.5` became `math.sqrt(dist)` since RISC-V has a native `fsqrt.s` instruction.

III- CSV Loading Problem

Assembly has no file I/O or CSV parsing. We realized `generate.py` was provided, a Python script that reads the CSV once and bakes all N particles into `mydata.S` as `.float` constants at compile time, included via:

```
.include "mydata.S"
```

This gave us all N bodies' positions, velocities, and masses as pre initialized float arrays with labels `p_x`, `v_x`, `mass_grid`, etc.

IV- Environment Setup

On Mac, Homebrew's QEMU only ships the system emulator, not the user-mode `qemu-riscv64` needed. One group member's toolchain also did not support the required instructions. We standardized on Docker with Ubuntu 22.04, eliminating version mismatches entirely.

The preferred setup was 32-bit RISC-V since it simplifies stack operations. Our Docker environment only had the 64-bit toolchain, so we shifted to `riscv64`. This required `ld/sd` instead of `lw/sw` for stack pointer operations. Everything else

was unaffected, `.float` arrays still used `flw/fsw` normally and the 64-bit shift was ultimately not an issue.

V- Template.S

We were provided a template handling the mmap architecture, opening `shared.mem`, mapping it to memory, and providing a pointer in `s1`. Our job was to implement the physics functions and plug them into `main_loop`.

V-A File Extension Case Sensitivity

Assembly files with lowercase `.s` skip the C preprocessor, meaning `.include` directives do not work. All assembly files must use uppercase `.S`.

VI- Writing the Physics Functions

We wrote three functions: `kick()`, `drift()`, and `calculate_acc()`.

`kick()` and `drift()` are $O(N)$ loops. The key lesson: float constants (`dt`, `0.5`) must be loaded *before* array addresses, otherwise `la` reuses `t0` and overwrites the float value. `calculate_acc()` is $O(N^2)$. Two critical bugs:

- **Stack too small:** -64 bytes insufficient for 10 saved registers (`s0-s9`). Increased to -80 bytes. `s8/s9` were corrupting silently.
- **softening² overwritten:** Stored in `ft9`, but the mass load inside the inner loop reused `ft9`. Fixed by copying to `ft11`, a dedicated protected register — once outside both loops.

VII- Memory Mapped File & GUI Connection

Since the simulation runs under QEMU with no direct Python interop, a memory-mapped file (`shared.mem`) acts as shared memory. When assembly writes to it, Python sees changes instantly.

VII-A Setup

Three syscalls set up shared memory: `openat` to create `shared.mem`, `ftruncate` to pre allocate its size, and `mmap` to map it. The base address is stored in `s1` throughout the main loop.

VII-B Memory Layout

All offsets are computed dynamically so the implementation works for any N .

TABLE I
SHARED MEMORY LAYOUT (SHARED.MEM)

Offset	Content
Byte 0	Sync flag
4 to $N \times 4 + 3$	$p_x[0..N-1]$ (float32)
$N \times 4 + 4$ to $N \times 8 + 3$	$p_y[0..N-1]$ (float32)
$N \times 8 + 4$ to $N \times 12 + 3$	$p_z[0..N-1]$ (float32)

VII-C Synchronization

A race condition existed where Python could read `shared.mem` mid-write, resulting in corrupted particle positions. This was resolved using a single flag byte at offset 0. After writing all N positions, assembly sets the flag to 1 and busy-waits until it returns to 0. Python polls the flag, reads all three position arrays once it sees 1, then resets it to 0, guaranteeing only one process accesses shared memory at a time.

VII-D Issues Faced

- **Bus error:** `shared.mem` was 0 bytes. Fixed by adding `ftruncate` inside `_start`.
- **s1 corruption:** `mmap` pointer overwritten by physics functions. Fixed by saving/restoring `s1` around every call.
- **pygame not installed:** Fixed with `pip3 install pygame`.
- **Stars not moving visibly:** Expected limitation of $O(N^2)$ physics through QEMU emulation, significantly slower than native hardware.

VIII- QEMU Profiling

To profile a single iteration, we created `template_copy.s`, a copy of `template.s` with `write_done` jumping to `exit_gracefully` instead of looping:

```
write_done:
    j exit_gracefully # one iteration only
    # j main_loop     # commented out
```

Three plugins were run against `template_copy`:

TABLE II
TOTAL INSTRUCTION COUNT (LIBINSN.SO)

Metric	Value
Total instructions (1 iteration)	6,245,875
Unique instruction encodings	3,172

TABLE III
CACHE BEHAVIOR (LIBCACHE.SO)

Type	Accesses	Misses	Miss Rate
Data	1,809,162	544	0.0301%
Instruction	6,236,752	304	0.0049%

TABLE IV
INSTRUCTION TYPE BREAKDOWN (LIBINSN_PROFILE_V2.SO)

Category	Count	Percentage
Floating-Point	4,322,719	69.21%
Other	1,824,786	29.22%
Branch	94,922	1.52%
Load	2,968	0.05%
Store	480	0.01%
Vector	0	0.00%
Total	6,245,875	100%

The 69% floating-point share confirms the bottleneck is pure math computation. The near-zero cache miss rate (0.03%) shows sequential array access has excellent spatial locality.

IX- SIMD Vector Exercises

Three programs using RISC-V Vector Extension (RVV), compiled with `-march=rv64gcv`, run with `-cpu rv64, v=true, vlen=128`:

- **Clamp:** `vmsgt` creates a mask of values > 255 , `vmerge` selects 255 or the original — branchless conditional selection.
- **Negative Count:** `vmslt` creates a mask of negatives, `vcpop` counts 1-bits instantly — no per-element branch needed.
- **Dot Product:** `vmul` multiplies all pairs at once, `vredsum` collapses the vector to a single sum in one instruction.

With 128-bit registers and 32-bit integers, 4 elements are processed per instruction. The same code scales to $16\times$ throughput by increasing `vlen` to 512 without any code changes.

Register Allocation — kick(), drift(), calculate_acc()

RISC-V Assembly — N-Body Simulation

2

kick()

Stack: -32 bytes (s0, s1, ra)

Reg	Type	Holds
s0	saved	loop counter i
s1	saved	N (loop limit)
t0	temp	base address of v_x
t1	temp	base address of a_x
t2	temp	base address of v_y
t3	temp	base address of a_y
t4	temp	base address of v_z
t5	temp	base address of a_z
t6	temp	byte offset = i * 4
ft0	float	dt * 0.5 (half timestep)
ft1	float	0.5 constant
fa0	float	current velocity v[i]
fa1	float	a[i] * dt * 0.5 (temp)
a0	arg	address of v[i]
a1	arg	address of a[i]

drift()

Stack: -32 bytes (s0, s1, ra)

Reg	Type	Holds
t0	temp	base address of p_x
t1	temp	base address of v_x
t2	temp	base address of p_y
t3	temp	base address of v_y
t4	temp	base address of p_z
t5	temp	base address of v_z
t6	temp	loop counter i
fa0	float	dt * 0.5 (half timestep)
fa1	float	v[i] * dt * 0.5 (temp)
fa2	float	p[i] current value
fa3	float	0.5 constant
a0	arg	temp address for constants
a1	arg	N — set outside loop
a2	arg	byte offset = i * 4
a3	arg	temp address register

calculate_acc()

Stack: -80 bytes (s0-s9)

Phase 1: Reset Loop

Reg	Type	Holds
s0	saved	loop counter i
s1	saved	N = 300
t0-t2	temp	base addr of a_x, a_y, a_z
t3	temp	byte offset = i * 4
t4	temp	temp address for stores
ft11	float	0.0 constant (zeroing arrays)

Phase 2: Force Loop

Reg	Type	Holds
s0	saved	outer loop counter i
s1	saved	inner loop counter j
s2	saved	N = 300
s3-s5	saved	base addr p_x, p_y, p_z
s6-s8	saved	base addr a_x, a_y, a_z
s9	saved	base addr mass_grid
ft0	float	dx = p_x[j] - p_x[i]
ft1	float	dy = p_y[j] - p_y[i]
ft2	float	dz = p_z[j] - p_z[i]
ft3	float	dist_sq = dx ² +dy ² +dz ² +soft ²
ft4	float	dist = sqrt(dist_sq)
ft5	float	dist_cubed = dist ² × dist
ft6	float	G / dist_cubed
ft7	float	mass[i]
ft8	float	G constant (loaded once)
ft9	float	temp (reused for acc reads)
ft10	float	temp force component
ft11	float	softening ² — protected reg
fa0	float	mass[j]
t0-t1	temp	i*4 and j*4 offsets
t2-t3	temp	temp address registers
t4	temp	temp addr for acc update